

1. 加锁协议的基本思想是什么?它如何确保正确执行并发事务?采用锁定协议后必须解决的问题是什么?

2. 传统数据管理经历了人工管理、文件系统、数据库系统等阶段,在这一过程中,数据独立性越来越高,为什么数据管理系统的数据独立性越高越好?(阐明数据的两种独立性的含义和意义)

3. 加锁是现代数据库系统中事务并发管理的重要技术,试回答下述问题:

(1) 已有的 (S, X)、(S, U, X) 锁能解决事务并发中的死锁问题么?为什么?

(2) (S, U, X) 锁的相容矩阵如下图,为什么已经加了 U 锁,不允许其它事务申请加 U 锁?如果允许会出现什么情况?

其它事务已拥有的锁				
锁 请 求		S	U	X
	S	Y	Y	N
	U	Y	N	N
	X	N	N	N

4. 假设运行记录与数据库的存储磁盘有独立失效模式,介质失效恢复时,对运行记录中上一检查点以前的已提交事务应该 redo 否?为什么?什么时候可以清空 CTL 表?

5. 某网上商品销售系统的业务流程如下:

(1) 将客户的订单记录(订单号, 客户 ID, 商品 ID, 购买数量)写入订单表;

(2) 将库存表(商品 ID, 库存量)中订购商品的库存量减去该商品的购买数量。

针对上述业务流程,引入如下伪指令:

- 将商品 A 的订单记录插入订单表记为 $I(A)$;
- 读取商品 A 的库存量到变量 x , 记为 $x = R(A)$;
- 变量 x 值写入商品 A 中的库存量, 记为 $W(A, x)$ 。
- 则客户 i 的销售业务伪指令序列为: $I_i(A)$, $x_i = R_i(A)$, $x_i = x_i - a_i$, $W_i(A, x_i)$. 其中 a_i 为商品的购买数量。

假设当前库存量足够,不考虑发生修改后库存量小于 0 的情况。若客户 1、客户 2 同时购买同一种商品时,可能出现的执行序列为:

$I_1(A)$, $I_2(A)$, $x_1 = R_1(A)$, $x_2 = R_2(A)$, $x_1 = x_1 - a_1$, $W_1(A, x_1)$, $x_2 = x_2 - a_2$, $W_2(A, x_2)$ 。

请回答以下问题:

(1) 客户此时会出现什么问题?

(2) 为了解决上述问题,引入共享锁指令 $S\text{Lock}(A)$ 和独占锁指令 $X\text{Lock}(A)$ 对数据 A 进行加锁,解锁指令 $\text{Unlock}(A)$ 对数据 A 进行解锁,客户 i 的加锁指令用 $S\text{Lock}_i(A)$ 表示,其他类同。插入订单表的操作不需要引入锁指令。请补充上述执行序列,使其满足 2PL 协议,并使持有锁的时间最短。

6. 设数据库中包含下列关系:

course(cno, cname, dame)/*课程号, 课程名, 开课系名*/

enroll(sid, grade, cno, dname1, sectno, pname, dname2)/*学号, 成绩, 课程号, 开课系名, 分班号, 任课教师名, 任课教师所在系名*/

student(sid, sname, sex, age, year, gpa)/*学号, 姓名, 性别, 年龄, 年级, 成绩点*/

试编写一个触发器, 监视上面数据库中 enroll 表上的 INSERT 操作, 对每条 INSERT 语句, 判断其插入元组的 grade 值是否小于 3.0, 将这样的元组自动插入到 failedcourse 表中(设 failedcourse 表与 enroll 表的模式完全相同)。

1. 加锁协议的基本思想是什么?它如何确保正确执行并发事务?采用锁定协议后必须解决的问题是什么?

加锁协议的基本思想是“并发事务在对同一个数据对象进行操作之前, 会向系统发送一个锁定操作对象的请求。在事务的锁请求被批准后, 它才能对对象进行一定的控制。”

由于在该事务释放锁之前, 其他事务无法获取数据对象的锁请求并对其进行操作, 从而避免了访问冲突, 保证了并发事务的正确执行。

采用锁定协议后, 可能会出现活锁、死锁等问题, 其中必须解决的问题是事务之间的循环等待造成的死锁问题。

2. 传统数据管理经历了人工管理、文件系统、数据库系统等阶段, 在这一过程中, 数据独立性越来越高, 为什么数据管理系统的数据独立性越高越好? (阐明数据的两种独立性的含义和意义)

数据独立性是指建立在数据的逻辑结构和物理结构分离的基础上, 用户只需要以简单的逻辑结构操作数据而无需考虑数据实际的物理结构, 中间的转换工作由数据库管理系统实现。数据独立性分为数据的物理独立性和数据的逻辑独立性。

✓ 数据的物理独立性:

■ 含义: 是指用户的应用程序与存储在磁盘上的数据库中数据是相互独立的。用户程序不需要了解实际的物理结构, 只要处理数据的逻辑结构即可。

■ 意义:

[1] 可以实现数据的物理存取与应用程序实现相分离。

[2] 当数据的物理存储结构与存取方法的改变时, 不要求修改应用程序。

[3] 使初步数据共享成为可能, 只要知道数据存取结构, 不同程序可共用同一数据文件。

✓ 数据的逻辑独立性的意义:

■ 含义: 用户的应用程序与数据库的逻辑结构是相互独立的, 即, 当数据的逻辑结构改变时, 用户程序也可以不变。

■ 意义:

[1] 将数据的使用与数据的逻辑结构相分离。

3. 加锁是现代数据库系统中事务并发管理的重要技术, 试回答下述问题:

(1) 已有的 (S, X)、(S, U, X) 锁能解决事务并发中的死锁问题么? 为什么?

(2) (S, U, X) 锁的相容矩阵如下图, 为什么已经加了 U 锁, 不允许其它事务申请加 U 锁? 如果允许会出现什么情况?

其它事务已拥有的锁				
锁 请 求		S	U	X
	S	Y	Y	N
	U	Y	N	N
	X	N	N	N

(1) 不能解决并发事务中的死锁问题。当一个事物 A 占用数据对象 a 的 X 锁, 事务 B 占用数据对象 b 的 X 锁, 事务 A 和事务 B 又分别申请数据对象 b 和数据对象 a 的锁, 在 (S, X) 和 (S, U, X) 锁中, 均无法获准, 需要等待对方事务释放锁, 而进入等待状态则无法释放自己所占用的锁, 从而陷入循环等待, 即死锁。

(2) U 锁表示事务对数据对象进行更新的操作，但事实上在最后写入阶段事务需要再将其升级为 X 锁才行。如果此时允许其他事务再申请 U 锁，导致最终写操作时无法申请到 X 锁而导致死锁。比如事务 A 想将 U 锁升级为 X 锁进行数据写操作时，由于存在其他事物对数据对象的 U 锁，而无法升级为 X 锁，从而导致死锁。

4. 假设运行记录与数据库的存储磁盘有独立失效模式，介质失效恢复时，对运行记录中上一检查点以前的已提交事务应该 redo 否？为什么？什么时候可以清空 CTL 表？

介质失效指磁盘发生故障，数据库受损，如磁盘、刺头破损。

介质失效后应该在新介质中加载最近后备副本，并用档案存储器内运行记录中的后像，redo 后备副本以后提交的所有更新事物。如果检查点比后备副本要新，则对后备副本以后，检查点以前的事物也应该 redo。

在取后备副本后，以前的运行记录就失去价值，可以清空 CTL 表。

5. 某网上商品销售系统的业务流程如下：

(1) 将客户的订单记录（订单号，客户 ID，商品 ID，购买数量）写入订单表；

(2) 将库存表（商品 ID，库存量）中订购商品的库存量减去该商品的购买数量。

针对上述业务流程，引入如下伪指令：

- 将商品 A 的订单记录插入订单表记为 $I(A)$ ；
- 读取商品 A 的库存量到变量 x ，记为 $x = R(A)$ ；
- 变量 x 值写入商品 A 中的库存量，记为 $W(A, x)$ 。
- 则客户 i 的销售业务伪指令序列为： $I_i(A)$ ， $x_i = R_i(A)$ ， $x_i = x_i - a_i$ ， $W_i(A, x_i)$ 。其中 a_i 为商品的购买数量。

假设当前库存量足够，不考虑发生修改后库存量小于 0 的情况。若客户 1、客户 2 同时购买同一种商品时，可能出现的执行序列为：

$I_1(A)$ ， $I_2(A)$ ， $x_1 = R_1(A)$ ， $x_2 = R_2(A)$ ， $x_1 = x_1 - a_1$ ， $W_1(A, x_1)$ ， $x_2 = x_2 - a_2$ ， $W_2(A, x_2)$ 。

请回答以下问题：

(1) 客户此时会出现什么问题？

(2) 为了解决上述问题，引入共享锁指令 SLock(A) 和独占锁指令 XLock(A) 对数据 A 进行加锁，解锁指令 Unlock(A) 对数据 A 进行解锁，客户 i 的加锁指令用 SLock _{i} (A) 表示，其他类同。插入订单表的操作不需要引入锁指令。请补充上述执行序列，使其满足 2PL 协议，并使持有锁的时间最短。

(1) 会出现的问题：客户购买后写入的库存量值被错误地覆盖，最后的库存量只体现了客户的购买量，但不能体现客户 1 已购买，属于丢失修改造成的数据库不一致性。

(2) 重写后的序列：

$I_1(A)$ ， $I_2(A)$ ，XLock₁(A)， $x_1 = R_1(A)$ ， $x_1 = x_1 - a_1$ ， $W_1(A, x_1)$ ，Unlock₁(A)，
XLock₂(A)， $x_2 = R_2(A)$ ， $x_2 = x_2 - a_2$ ， $W_2(A, x_2)$ ，Unlock₂(A)

6. 设数据库中包含下列关系：

course(cno, cname, dame)/*课程号，课程名，开课系名*/

enroll(sid, grade, cno, dname1, sectno, pname, dname2)/*学号，成绩，课程号，开课系名，分班号，任课教师名，任课教师所在系名*/

student(sid, sname, sex, age, year, gpa)/*学号，姓名，性别，年龄，年级，成绩点*/

试编写一个触发器，监视上面数据库中 enroll 表上的 INSERT 操作，对每条 INSERT 语句，判断其插入元组的 grade 值是否小于 3.0，将这样的元组自动插入到 failedcourse 表中(设 failedcourse 表与 enroll 表的模式完全相同)。

```
CREATE TRIGGER insert_grade_check
AFTER INSERT ON enroll
REFERENCING NEW TABLE AS NE
FOR EACH STATEMENT
WHEN (EXISTS(SELECT * FROM NE
              WHERE grade<3.0))
INSERT
    INTO failedcourse
    SELECT * FROM NE
    WHERE NE.grade<3.0
```